Introduction to Recursion

1. What is Recursion

Recursion is a programming technique where a method calls itself to solve a problem. It's like breaking down a big problem into smaller, similar problems until we reach a simple case that we can solve directly.

Key Components of Recursion

Base Case: The condition that stops the recursion. Without this, the method would call itself forever.

Recursive Case: The part where the method calls itself with a modified parameter, moving closer to the base case.

Simple Example - Factorial

```java
public static int factorial(int n) {
    // Base case
    if (n == 0 || n == 1) {
        return 1;
    }
    // Recursive case
    return n * factorial(n - 1);
}
```

How it works:

- factorial(5) = 5 × factorial(4)
- factorial(4) = 4 × factorial(3)
- factorial(3) = 3 × factorial(2)
- factorial(2) = 2 × factorial(1)
- factorial(1) = 1 (base case reached)

Another Example - Counting Down

```java
public static void countdown(int n) {
    // Base case
    if (n == 0) {
        System.out.println("Done!");
        return;
    }
    // Print current number
    System.out.println(n);
    // Recursive case
    countdown(n - 1);
}
```

2. Why We Required Recursion

Makes Complex Problems Simple

Some problems are naturally recursive and are much easier to solve using recursion than loops.

Tree and Graph Problems

Recursion is perfect for problems involving trees, graphs, and hierarchical structures.

```java
// Example: Finding maximum value in a binary tree
public static int findMax(TreeNode root) {
    // Base case
    if (root == null) {
        return Integer.MIN_VALUE;
    }

    // Recursive case
    int leftMax = findMax(root.left);
    int rightMax = findMax(root.right);

    return Math.max(root.val, Math.max(leftMax, rightMax));
}
```

Divide and Conquer Algorithms

Many efficient algorithms use recursion to break problems into smaller parts.

```java
// Example: Binary Search
public static int binarySearch(int[] arr, int target, int left, int right) {
    // Base case
    if (left > right) {
        return -1; // Not found
    }

    int mid = left + (right - left) / 2;

    // Found the target
    if (arr[mid] == target) {
        return mid;
    }

    // Recursive cases
    if (arr[mid] > target) {
        return binarySearch(arr, target, left, mid - 1);
    } else {
        return binarySearch(arr, target, mid + 1, right);
    }
}
```

Mathematical Sequences

Perfect for calculating mathematical sequences like Fibonacci numbers.

```java
public static int fibonacci(int n) {
    // Base cases
    if (n == 0) return 0;
    if (n == 1) return 1;

    // Recursive case
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Clean and Readable Code

Recursive solutions often match the mathematical definition of problems, making code more intuitive.

3. Practice Basic Questions on Recursion

Example 1: Sum of Natural Numbers

```java
public static int sumOfNumbers(int n) {
    // Base case
    if (n == 1) {
        return 1;
    }
    // Recursive case
    return n + sumOfNumbers(n - 1);
}

// Usage
public static void main(String[] args) {
    System.out.println(sumOfNumbers(5)); // Output: 15
}
```

Example 2: Power Function

```java
public static int power(int base, int exponent) {
    // Base case
    if (exponent == 0) {
        return 1;
    }
    // Recursive case
    return base * power(base, exponent - 1);
}

// Usage
public static void main(String[] args) {
    System.out.println(power(2, 3)); // Output: 8
}
```

Example 3: Reverse a String

```java
public static String reverseString(String str) {
    // Base case
    if (str.length() <= 1) {
        return str;
    }
    // Recursive case
    return reverseString(str.substring(1)) + str.charAt(0);
}

// Usage
public static void main(String[] args) {
    System.out.println(reverseString("hello")); // Output: olleh
}
```

Example 4: Check if String is Palindrome

```java
public static boolean isPalindrome(String str, int start, int end) {
    // Base case
    if (start >= end) {
        return true;
    }

    // Check if characters match
    if (str.charAt(start) != str.charAt(end)) {
        return false;
    }

    // Recursive case
    return isPalindrome(str, start + 1, end - 1);
}

// Usage
public static void main(String[] args) {
    String word = "racecar";
    System.out.println(isPalindrome(word, 0, word.length() - 1)); // Output: true
}
```

Example 5: Array Sum

```java
public static int arraySum(int[] arr, int index) {
    // Base case
    if (index >= arr.length) {
        return 0;
    }
    // Recursive case
    return arr[index] + arraySum(arr, index + 1);
}

// Usage
public static void main(String[] args) {
    int[] numbers = {1, 2, 3, 4, 5};
    System.out.println(arraySum(numbers, 0)); // Output: 15
}
```

Quiz Questions

Question 1: What are the two essential components of a recursive function? a) Loop and condition b) Base case and recursive case c) Input and output d) Method and parameter

Answer: b) Base case and recursive case

Question 2: What happens if a recursive function doesn't have a base case? a) The program runs faster b) The program stops immediately c) The program runs into infinite recursion d) The program compiles with errors

Answer: c) The program runs into infinite recursion

Question 3: What will be the output of factorial(4)? a) 12 b) 24 c) 16 d) 20

Answer: b) 24

Question 4: In the fibonacci sequence, what are the first two base cases typically? a) fibonacci(0) = 0, fibonacci(1) = 1 b) fibonacci(0) = 1, fibonacci(1) = 1 c) fibonacci(0) = 1, fibonacci(1) = 2 d) fibonacci(1) = 0, fibonacci(2) = 1

Answer: a) fibonacci(0) = 0, fibonacci(1) = 1

Question 5: Which of the following problems is best solved using recursion? a) Finding the sum of two numbers b) Printing numbers from 1 to 10 c) Traversing a binary tree d) Swapping two variables

Answer: c) Traversing a binary tree

Practice Coding Questions

Question 1: Write a recursive method to find the greatest common divisor (GCD) of two integers using the Euclidean algorithm.

Question 2: Create a recursive method that counts the number of digits in a positive integer.

Question 3: Write a recursive method to print all possible binary strings of length n.

Question 4: Implement a recursive method to find the minimum element in an array.

Question 5: Write a recursive method that calculates the sum of digits of a positive integer.